QUANSER
INNOVATE·EDUCATE

Lab Procedure
Load Cell

Introduction

Ensure the following:

1. You have reviewed the **Application Guide – Load Cell**
2. Make sure the **Mechatronic Sensors Trainer** is out of its case and plug in the load cell cable. Ensure the red wire is on the left side of the socket.
3. Power up the **Mechatronic Sensors Trainer** by connecting the provided USB cable to the PC.
    a. The LED next to the USB C port will stay white after the touch screen starts loading.
    b. The load screen with the Quanser Logo should disappear after a few seconds and you should see some evenly spaced crosses on a black background on the touch screen.
4. You have 3-5 small objects with known weights. They will be used to calibrate the load cell. One example is your mobile phone, by looking up the exact model, their rough weight can be found.

Analyzing Raw Load Cell Data

1. Launch Visual Studio Code or your preferred Python IDE. If using Visual Studio Code, click on **`File > Open Folder`**…, and browse to the directory containing the python files and lab procedure for the Encoders lab (…/sensors_trainer/1_fundamentals/load_cell/hardware/python). Open this directory.
2. Open **load_cell_calibrate.py**. Run this script using the  button or the terminal. This will open a plot displaying raw load cell data. Gently tap the load cell, does plot display the expected results?
3. In the Set-up region of the scrip, change the **duration** parameter to 5. Run the script and let it complete after 5 seconds. The mean and standard deviation of the raw data over this duration is printed. Why is this information important?
4. Add one object to the load cell and run the scripts for 5 seconds again. How does mean and standard deviation change with the added weight?
5. Stack more objects on the load cell and repeat the process. Make sure the stacked objects does not exceed the weight limit (3 kg). What trend do you notice?

Load Cell Calibration

1. In the previous section, we have observed the trend of load cell data with increasing weight. In this section, we want to quantitatively define this trend. In the same directory, open **load_cell_calibrate.py**. You will need to complete the functions **process()** and **calibrate()**.

2. As observed in the previous sections, raw data from the load cell can be noisy. In the **process()** function, we want to output the mean of the data over a period specified by **weightTime**. This function should collect raw load cell data until the duration is met, then return the mean. Complete the function.

3. Run this script using the ▷ button. You will be prompted to enter the weight your selected object. After specified duration, the output of **process**() should be printed in the terminal. Verify that **process()** functions as expected.

4. Stop the script using **`Ctrl+C`** when you're satisfied with the output from **process()**.

5. Now we complete the **calibrate()** function. The input **recordedData** is a list of recorded weight and voltage pairs. As observed in the previous section, the raw data from the load cell increases linearly with weight of the object, we can assume the weight and voltage satisfy the following:

   $$\text{Weight} = a \times \text{Voltage} + b$$

   The parameters that define this linear relationship, **(a, b)**, should be the output of **calibrate()**. You can utilize common libraries such as Numpy's **polyfit()**.

6. Once both functions are completed, run the script using the ▷ button. When prompted, place a known weight on the load cell and enter the weight value. Repeat this step for multiple know weights.

7. When finished, type **'q'** or press **Enter** on an empty line to end the calibration session. The calibration coefficients are printed in the terminal for reference. Save these values for later use in the measurement phase.

Measuring Weight with Load Cell

1. In the same directory, open **load_cell_calibrate.py**. In this section, you will use the calibrated parameters to convert live load cell readings into corresponding weight values.

2. In the main load cell data collection loop, implement a moving average filter to reduce signal noise and improve measurement stability. Use Python's **deque** object from the **collections** module to efficiently maintain a fixed-length data buffer.

3. Apply the linear model using the calibrated parameters to estimate the applied weight from the filtered voltage readings.

4. Once the main loop is completed, run the script. Use the linear calibration equation derived from Section B to estimate the applied weight from the filtered voltage readings. Discuss whether the observed error is within the acceptable tolerance range for your system.

5. Once all measurements are completed, power OFF the **Mechatronic Sensors Trainer** by disconnecting its USB C cable, disconnect any external sensors and pack them along the base and the USB cables in the provided case.